

# File Handling and Data Processing in Python

## Contents

|           |                                         |          |
|-----------|-----------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                     | <b>3</b> |
| <b>2</b>  | <b>Opening a File</b>                   | <b>3</b> |
| <b>3</b>  | <b>Reading a Text File</b>              | <b>3</b> |
| 3.1       | Why use with? . . . . .                 | 3        |
| <b>4</b>  | <b>Reading Line by Line</b>             | <b>4</b> |
| 4.1       | Why line by line? . . . . .             | 4        |
| <b>5</b>  | <b>EOF Concept</b>                      | <b>4</b> |
| <b>6</b>  | <b>Passwords and Hash Concept</b>       | <b>4</b> |
| <b>7</b>  | <b>Stripping Data</b>                   | <b>4</b> |
| <b>8</b>  | <b>File as Database (Simple Idea)</b>   | <b>5</b> |
| <b>9</b>  | <b>Guess Game Record System</b>         | <b>5</b> |
| 9.1       | Why store score in file? . . . . .      | 5        |
| <b>10</b> | <b>Updating File Data</b>               | <b>5</b> |
| <b>11</b> | <b>Append Mode</b>                      | <b>5</b> |
| <b>12</b> | <b>CSV Files</b>                        | <b>6</b> |
| <b>13</b> | <b>Reading CSV File</b>                 | <b>6</b> |
| <b>14</b> | <b>next() Function</b>                  | <b>6</b> |
| <b>15</b> | <b>Processing CSV Rows</b>              | <b>6</b> |
| <b>16</b> | <b>Calculating Percentage</b>           | <b>6</b> |
| <b>17</b> | <b>Why File Handling is Important</b>   | <b>6</b> |
| <b>18</b> | <b>Bitwise Operators (Introduction)</b> | <b>7</b> |

|                               |   |
|-------------------------------|---|
| 19 AND Operator (&)           | 7 |
| 20 OR Operator (—)            | 7 |
| 21 Right Shift ( >>)          | 7 |
| 22 Bitwise Trick: Even or Odd | 8 |
| 23 Rule                       | 8 |
| 24 Real-world Use of Bitwise  | 8 |
| 25 Final Summary              | 8 |

# 1 Introduction

Till now, we worked with:

- Variables
- Conditions
- Loops
- Functions
- Modules

Now we move to something very powerful:

## File Handling

File handling allows Python programs to:

- Store data permanently
- Read external data
- Process large datasets

This is how real applications work.

# 2 Opening a File

Basic syntax:

```
open("filename", "mode")
```

Modes:

- "r" → read
- "w" → write
- "a" → append

# 3 Reading a Text File

```
with open("mytext.txt", "r") as file:  
    content = file.read()  
  
print(content)
```

## 3.1 Why use with?

`with` automatically closes the file.

Without `with`, file must be closed manually.

## 4 Reading Line by Line

```
with open("new.txt", "r") as file:
    for line in file.readlines():
        print(line)
```

### 4.1 Why line by line?

Useful when:

- File is large
- Data is structured row-wise

## 5 EOF Concept

EOF means:

End Of File

It indicates no more data is left to read.

## 6 Passwords and Hash Concept

Example idea:

```
wifi_password = "2123efa02364"
```

This is called a:

Hashed value

Instead of storing real password, systems store hashed versions for security.

## 7 Stripping Data

```
cleaned_pass = password.strip()
```

Removes:

- Spaces
- Newline characters ()

## 8 File as Database (Simple Idea)

We can store:

- game scores
- logs
- records

inside files.

## 9 Guess Game Record System

```
with open("guess_game_record.txt", "r") as file:  
    current_min_guess = file.read()
```

### 9.1 Why store score in file?

So that:

- Data persists after program closes
- We can track best performance

## 10 Updating File Data

```
with open("guess_game_record.txt", "w") as file:  
    file.write(str(counter))
```

Mode:

- "w" → overwrites file

## 11 Append Mode

```
with open("web_logs.txt", "a") as file:  
    file.writelines(user_inputs)
```

Mode:

- "a" → adds data without deleting old content

Used for:

- logs
- history tracking

## 12 CSV Files

CSV = Comma Separated Values

Example:

```
Ram, 80, 75, 90
Hari, 60, 70, 65
```

## 13 Reading CSV File

```
import csv

with open("class6_marks.csv", "r") as file:
    reader = csv.reader(file, delimiter=",")
    header = next(reader)
```

## 14 next() Function

Skips first row (usually header)

## 15 Processing CSV Rows

```
for row in reader:
    name = row[0]
    marks = row[1:]
```

## 16 Calculating Percentage

```
total = 0

for mark in marks:
    total += int(mark)

percentage = total / len(marks)
```

## 17 Why File Handling is Important

File handling is used in:

- Databases (basic idea)
- Logging systems
- Games

- Analytics
- Web applications

## 18 Bitwise Operators (Introduction)

Python supports bit-level operations.

Binary example:

```
5 = 101
3 = 011
```

## 19 AND Operator (&)

```
5 & 3
```

Rule:

1 AND 1 = 1, otherwise 0

Example:

```
101
011
---
001 => 1
```

## 20 OR Operator (—)

```
5 | 3
```

Rule:

1 OR 0 = 1

Example:

```
101
011
---
111 => 7
```

## 21 Right Shift ( >> )

```
5 >> 1
```

Binary:

```
101      10
```

Meaning:

Divide by 2 (integer division)

## 22 Bitwise Trick: Even or Odd

```
7 & 1
```

Binary:

```
7  = 0111
1  = 0001
-----
    0001
```

## 23 Rule

- Result = 1 → Odd number
- Result = 0 → Even number

## 24 Real-world Use of Bitwise

Used in:

- Graphics programming
- Compression
- Cryptography
- System-level programming

## 25 Final Summary

Today you learned:

- File reading
- File writing
- Append mode
- CSV processing
- Basic logging system
- Bitwise operations

This is a major step toward:

Real-world backend programming